XRAY TEST MANAGEMENT PLUGIN IN JIRA

Concepts, Workflow and Practical Implementation

Training programme	SDET Training, Veeva Systems
Subject	Xray test management for Jira (Cloud and Data Center)
Prepared by	Sai
Date	October 2026
Document version	1.0

Table of Contents

1. Introduction to XRAY 4
1.1 What is XRAY? 4
1.2 What is the Xray Plugin in Jira? 4
1.3 Why was Xray introduced? 5
1.4 What problems does Xray solve? 5
1.5 How does Xray help QA engineers and testers? 5
2. Features of Xray 6
2.1 Test Case Management 6
2.2 Test Execution Management 6
2.3 Test Planning 6
2.4 Test Reporting 6
2.5 Requirement Traceability 7
2.6 Test Coverage Analysis 7
2.7 Integration with Jira Issues 7
2.8 Manual and Automated Testing Support 7
3. Xray Issue Types and Their Hierarchical Structure 7
3.1 Test 8
3.2 Test Set 8
3.3 Test Plan 8
3.4 Test Execution 9
3.5 Precondition 9
3.6 Requirement / Story 9
3.7 Bug 10
3.8 Relationship Diagram 10
4. Understanding the Xray Testing Workflow 11
5. How to Create and Manage Tests in Xray 13
5.1 Creating a Test issue 13
5.2 Writing a proper test case 13
5.3 Adding test steps 13
5.4 Defining expected results 13
5.5 Linking Tests to requirements or user stories 14
5.6 Creating a Test Set 14
5.7 Adding Tests to a Test Set 14
5.8 Creating a Test Plan 14
5.9 Adding Tests to a Test Plan 14
5.10 Sample test case formats 15
6. Test Execution in Xray 15
6.1 What is Test Execution? 15
6.2 Creating a Test Execution 16
6.3 Adding Tests to an execution 16
6.4 Executing manual tests 16
6.5 Recording actual results 16
6.6 Updating test status 16
6.7 Handling failed test cases 16
6.8 Associating bugs with failed tests 17
6.9 Execution statuses 18

7. Requirement Traceability and Test Coverage 18
7.1 What is requirement traceability? 18
7.2 How Xray links requirements with Tests 18
7.3 Identifying requirements without test coverage 19
7.4 Analysing test coverage 19
7.5 Why traceability is important 19
7.6 Practical example: login module 20
8. Xray Reports and Dashboards 20
8.1 How QA teams use the reports 21
9. Real-Time Practical Case Study: E-Commerce Application 22
9.1 Project setup 22
9.2 Sample Jira user stories 22
9.3 Preconditions 22
9.4 Sample test cases 23
9.5 Test Sets 24
9.6 Test Plan 24
9.7 Test Executions 24
9.8 Sample execution results (SHOP-72) 24
9.9 Defect creation and linking 25
9.10 Requirement traceability result 25
9.11 Complete workflow summary 26
10. Xray vs Traditional Jira Testing 26
10.1 Advantages of Xray for professional QA teams 27
11. Best Practices for Using Xray 28
12. Conclusion 28
References 30

1. Introduction to XRAY

1.1 What is XRAY?

Xray is a test management tool built on top of Jira. It lets a team design test cases, group them, run them, record results and measure test coverage without leaving Jira. In older articles and in this document's title the name is often written as XRAY; the product name used by its vendor is Xray, and that spelling is used in the rest of this document.

Test management means planning what to test, writing and organising the test cases, running them, recording what happened and reporting the result to the team. Xray covers all of these activities and keeps them next to the requirements and defects that Jira already tracks. [1]

1.2 What is the Xray Plugin in Jira?

Jira calls add-ons "apps" (older material says "plugins"). Xray is a third-party app that is installed from the Atlassian Marketplace. Once installed and enabled for a project, it adds new issue types, screens, fields and reports that are designed for testing. It is available in two deployment models, and the behaviour described in this document is the same in both unless a difference is stated.

Aspect	Xray Cloud	Xray Data Center
Used with	Jira Cloud	Jira Data Center (documentation also covers the older Jira Server, which Atlassian has retired)
Documentation space	XRAYCLOUD space on docs.getxray.app	XRAY space on docs.getxray.app
Programmatic access	REST and GraphQL APIs	REST API
Administering test statuses	Jira Global Settings, Apps section, Xray Test Statuses [8]	Xray administration, Manage Test Statuses [9]
Built-in reports	Test Coverage, Traceability, Tests List, Test Sets List, Test Executions List, Test Plans List, Test Plans Metrics, Test Runs List	Test Coverage, Traceability, Test Plans Report, Test Executions Report and others [4][5]
Edition-specific features	Some features depend on the Xray plan	Some features, such as selecting several projects in the Test Plans Report, require Xray Enterprise [4]

Because report names, menu labels and optional features change between releases, the practical steps in this document describe the action to take (for example "add Tests to the Test Plan") and mention button names only where they are stable. Always follow the labels shown in your own Jira instance and check the documentation for your Xray version.

It is important to separate what Jira already does from what Xray adds:

Provided by native Jira	Provided by Xray
Issues, projects, boards, sprints, versions	Issue types Test, Precondition, Test Set, Test Plan and Test Execution
Issue types such as Story, Epic, Task and Bug	Test steps (action, data, expected result), Test Types (Manual, Cucumber, Generic)
Workflows, permissions, notifications, JQL, dashboards	Test Runs, execution statuses and the execution screen
Issue links	The "Test" link used for requirement coverage, and the coverage status calculation
Attachments and comments	Test-specific reports and dashboard gadgets; import of automated test results

1.3 Why was Xray introduced?

Jira was designed to track work such as stories, tasks and bugs. A Jira issue has one status for the whole issue and cannot, on its own, record that "step 3 of this test failed on Tuesday in Chrome". Teams therefore kept test cases in spreadsheets or in separate tools, and the link between a requirement, the tests that verify it and the defects found by those tests was maintained by hand. Xray was introduced to bring that information into Jira so that requirements, tests, results and defects are connected in one place.

1.4 What problems does Xray solve?

Problem in ordinary Jira or spreadsheet testing	How Xray addresses it
Test cases are scattered across files	Each test case is a Jira issue (Test) that can be searched with JQL and reused
No step-level results	A Test Run records a status, actual result and evidence for every step
No history of executions	Every execution is kept, so the latest and earlier results of a Test can be compared
Unknown requirement coverage	Requirements show a coverage status calculated from linked Tests
Hard to see progress	Test Plans, reports and gadgets show how many Tests passed, failed or are still to be run
Defects are not tied to tests	A Bug can be created from a failed Test Run and stays linked to that run and Test
Automation results live elsewhere	Results from CI tools can be imported into Xray

1.5 How does Xray help QA engineers and testers?

- A tester gets a clear list of what to run, with steps, test data and expected results in front of them.

- A QA engineer can reuse one Test in many executions instead of rewriting it for every release.
- A QA lead can see progress and coverage without collecting status from each tester.
- An SDET can connect automated test results to the same Test issues that manual testers use, so manual and automated results appear in one report.
- Developers and product owners can open a requirement in Jira and see its tests, results and defects.

2. Features of Xray

This section describes the main features. Each one is followed by a short example from a login feature of an online shop, which is developed further in Section 9.

2.1 Test Case Management

Each test case is stored as a Jira issue of type **Test**. A Test has a Test Type: **Manual** (a list of steps), **Cucumber** (a scenario written in Gherkin) or **Generic** (an unstructured definition, often used to represent an automated or exploratory test). Tests can be grouped with Test Sets or arranged in folders in the Test Repository, and they can be labelled, assigned components and versioned like any other Jira issue. [12]

Example: Test SHOP-24 "Login with valid credentials" is written once with four steps. It is then reused in every sprint and release instead of being copied.

2.2 Test Execution Management

A **Test Execution** is a Jira issue that collects the Tests to be run for a particular purpose, such as one sprint, one release or one environment. For every Test added to it, Xray creates a **Test Run**, which stores the result of that Test in that execution: status, actual result, evidence, defects, start and end time. In Xray, Tests are always run inside a Test Execution. [2]

Example: Test Execution SHOP-72 "Release 1.0 Full Regression (Chrome, Staging)" holds 15 Test Runs, one for each Test in the release.

2.3 Test Planning

A **Test Plan** defines the scope of testing for a release, sprint or test cycle. It lists the Tests that must be run and shows the latest result of each Test across the Test Executions that belong to it. The Test Plan issue type was added in Xray version 2; earlier versions used Test Executions as a substitute. [7]

Example: Test Plan SHOP-61 "ShopEasy Release 1.0 Test Plan" lists 15 Tests. A smoke execution and a full regression execution are both associated with it.

2.4 Test Reporting

Xray provides built-in reports and dashboard gadgets that summarise coverage, execution progress and defects. Which reports are available depends on the Xray edition and version, as described in Section 8. [4][5][10]

Example: The test lead opens the Test Plans report before the daily stand-up to see what percentage of the Tests in Test Plan SHOP-61 have a final result.

2.5 Requirement Traceability

Traceability is the ability to follow a requirement forwards to the Tests that verify it, the results of those Tests and the defects they found, and to follow a defect backwards to the requirement it affects. In Xray this is done by linking Tests to requirement issues (for example Stories or Epics). [6]

Example: Story SHOP-12 "User Login" is linked to Tests SHOP-24, SHOP-25 and SHOP-26. Opening the story shows these Tests and their latest results.

2.6 Test Coverage Analysis

Coverage analysis answers the question "is every requirement tested, and did the tests pass?". Xray calculates a coverage status for each requirement from the latest results of its linked Tests. The default statuses are OK, NOK, NOTRUN, UNKNOWN and UNCOVERED, explained in Section 7. [3]

Example: A new Story with no linked Tests is shown as UNCOVERED, which tells the team that testing for it has not been designed yet.

2.7 Integration with Jira Issues

Because Tests, Test Plans and Test Executions are Jira issues, the normal Jira tools apply to them: workflows, permissions, fix versions, labels, comments, attachments, JQL searches, boards and dashboards. Xray's own documentation shows how workflows can be used to review and approve Tests before they are executed. [7]

Example: A team gives Tests a workflow of Draft, In Review and Approved, and only approved Tests are added to Test Executions.

2.8 Manual and Automated Testing Support

Manual Tests are executed step by step in the Xray execution screen. Cucumber Tests hold Gherkin scenarios. Automated tests normally run in a continuous integration (CI) tool, and their results are imported into Xray, which creates Test Execution issues for the imported results. Result formats that Xray can import include JUnit, TestNG, Cucumber JSON and Xray JSON. [2]

Example: A Selenium and TestNG suite runs in a Jenkins job each night. The job imports the TestNG result file into Xray, and each automated test is matched to a Test issue, so manual and automated results are reported together.

3. Xray Issue Types and Their Hierarchical Structure

Five of the seven items below are issue types that Xray adds to Jira: Test, Test Set, Test Plan, Test Execution and Precondition. The other two, Requirement (usually a Story) and Bug, are normal Jira issue types. Two further terms are often confused with issue types and should be kept separate:

- **Test Run** is not an issue type. It is the record of one Test inside one Test Execution.

- **Execution status** (TODO, EXECUTING, PASS, FAIL, ABORTED) is a status of a Test Run or Test. It is not the Jira workflow status of an issue and it is not an issue type.

3.1 Test

Origin	Added by Xray
What it represents	One test case: what to verify and how to verify it. Test Types are Manual, Cucumber and Generic.
Why it is used	It is the basic reusable unit of testing. Everything else (sets, plans, executions) organises or runs Tests.
Relationships	Linked to requirements it covers; linked to Preconditions it needs; a member of any number of Test Sets; tracked by Test Plans; run through Test Executions, which create Test Runs.
Practical example	SHOP-24 "Login with valid credentials", Manual, four steps, linked to Story SHOP-12.

3.2 Test Set

Origin	Added by Xray
What it represents	A flat list of Tests grouped for a purpose, such as smoke, regression or one module. [12]
Why it is used	It allows many Tests to be selected together, for example to add them to a Test Execution or Test Plan in one step.
Relationships	A Test Set groups Tests. A Test can belong to many Test Sets, so a Test Set is a grouping, not a parent that owns its Tests. The Test Repository (a folder tree) is the alternative way to organise Tests; a Test sits in one folder.
Practical example	SHOP-51 "Smoke - Core Purchase Flow" groups the five most important Tests of the shopping journey.

3.3 Test Plan

Origin	Added by Xray
What it represents	The scope and progress of testing for a release, sprint or cycle.
Why it is used	It answers "what do we intend to test, and how far have we got?". It shows the latest result of each tracked Test, taken from the related executions. [7]
Relationships	Tracks a set of Tests (Tests can be added individually or from Test Sets). Test Executions can be associated with it. The same Test can appear in many Test Plans.
Practical example	SHOP-61 "ShopEasy Release 1.0 Test Plan" tracks 15 Tests and has two executions associated with it.

3.4 Test Execution

Origin	Added by Xray
What it represents	One round of running a chosen group of Tests, for example for one sprint, build or environment. It has fields such as fix version, revision, dates and test environments. [12]
Why it is used	It keeps the results of one round separate from other rounds, so history is preserved.
Relationships	Contains one Test Run for each Test added to it. Can be associated with a Test Plan. Defects are raised from its Test Runs.
Practical example	SHOP-72 "Release 1.0 Full Regression (Chrome, Staging)".
Note	Some Xray versions also provide a Sub-Test Execution issue type, which behaves like a Test Execution. It is optional and is not covered further here. [5]

3.5 Precondition

Origin	Added by Xray
What it represents	A condition that must be true before a Test can be run, such as required data or system state.
Why it is used	It is written once and reused. One Precondition can be linked to many Tests, which avoids repeating setup text in every Test.
Relationships	Linked to Tests (many-to-many).
Practical example	SHOP-41 "A registered, active customer account exists" is linked to all login Tests.

3.6 Requirement / Story

Origin	Native Jira issue type, for example Story, Epic or Task
What it represents	A feature or need that the software must satisfy. Any issue type can be treated as a requirement if the project is configured for it in the Xray settings.
Why it is used	It is the reason Tests exist. Linking Tests to it gives traceability and coverage status.
Relationships	Linked to Tests by the Xray "Test" link (a Test "tests" a requirement; a requirement "is tested by" a Test).
Practical example	SHOP-12 "As a registered customer, I want to log in so that I can see my orders."

3.7 Bug

Origin	Native Jira issue type
What it represents	A defect found in the product.
Why it is used	It records the problem and drives the fix. When it is raised from a failed Test Run, Xray keeps the connection to that run, so the failing Test and Test Execution are known.
Relationships	Linked to the Test Run (and therefore the Test Execution and Test) where it was found. It can also be linked to the affected requirement with a normal Jira link.
Practical example	SHOP-81 "Account is not locked after five failed login attempts", raised from the failed run of SHOP-26.

3.8 Relationship Diagram

The table and the diagram that follows show how these items are associated. The diagram is deliberately not drawn as a tree. Most relationships are many-to-many associations: one Test can cover several requirements, belong to several Test Sets, be tracked by several Test Plans and be run in many Test Executions. Dashed arrows show optional associations.

From	Association	To	Cardinality and notes
Test	covers ("tests")	Requirement / Story	Many-to-many
Test	requires	Precondition	Many-to-many
Test Set	groups	Test	Many-to-many; grouping only
Test Plan	tracks	Test	Many-to-many
Test Plan	can import Tests from	Test Set	Optional; a quick way to add many Tests
Test Execution	contains a Test Run for each	Test	Many-to-many over time; one run per Test per execution
Test Execution	is planned under	Test Plan	Optional; many executions per plan
Bug	is raised from	Test Run / Test Execution	Created when a Test Run fails
Bug	relates to	Requirement / Story	Optional Jira link

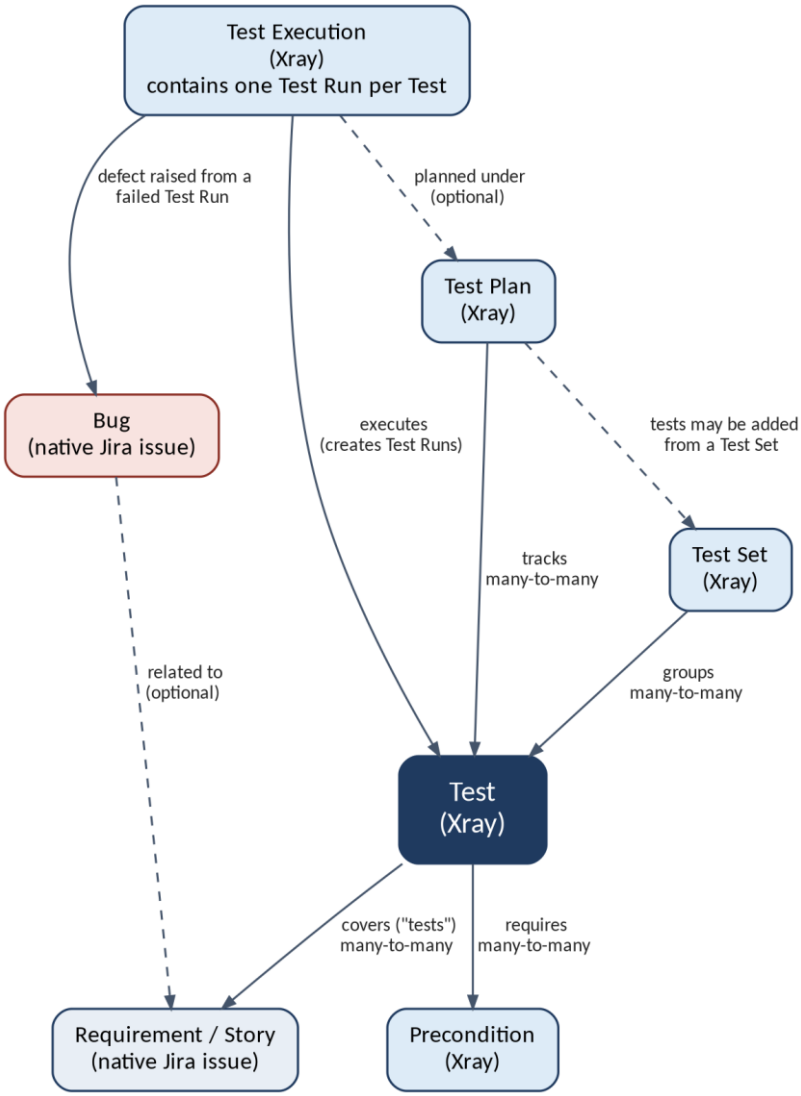


Figure 1: Associations between Xray entities and Jira issues

4. Understanding the Xray Testing Workflow

The workflow has twelve steps. Xray's documentation notes that the testing phases (plan, design, execute, report) are not always strictly sequential, and some steps are optional: a Precondition is only created when needed, and a Test Set or Test Plan can be skipped on small projects. [7] The order below is the one most teams follow.

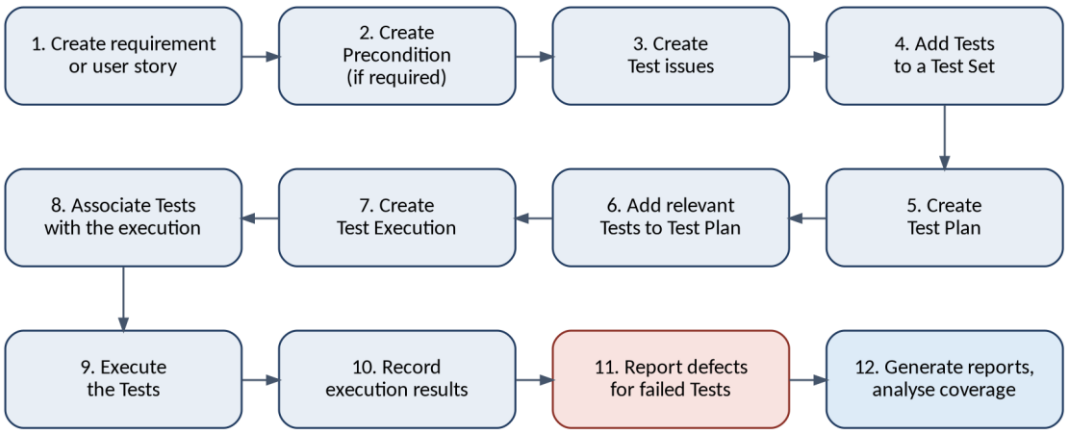


Figure 2: End-to-end Xray testing workflow

No.	Step	Purpose	How it connects to other steps
1	Create a requirement or user story in Jira	Defines what must be tested	Tests created in step 3 are linked to it
2	Create a Precondition if required	Records setup that must exist before a Test can run	Linked to Tests in step 3
3	Create Test issues	Designs the test cases, with steps and expected results	Linked to the requirement and Preconditions
4	Add Tests to a Test Set	Groups Tests by purpose (smoke, regression, module)	Used to select Tests quickly in steps 6 and 8
5	Create a Test Plan	Defines the scope for a release or sprint	Receives Tests in step 6
6	Add relevant Tests to the Test Plan	Decides which Tests are in scope	Tests can come from Test Sets
7	Create a Test Execution	Opens one round of testing	Can be associated with the Test Plan
8	Associate the required Tests with the execution	Creates a Test Run for each Test	Tests can be taken from the Test Plan or a Test Set
9	Execute the Test cases	Performs the steps and compares actual with expected results	Done inside the Test Runs of step 8
10	Record execution results	Stores status, actual result and evidence	Updates Test, requirement and Test Plan status
11	Report defects when Tests fail	Creates Bugs linked to the failed Test Runs	Defects appear in reports and in traceability
12	Generate reports and analyse coverage	Shows progress and gaps; supports release decisions	Uses all the data from steps 1 to 11

5. How to Create and Manage Tests in Xray

The procedures below assume that Xray is installed and that the Xray issue types are enabled for the project (a Jira or project administrator does this). Button and menu names can differ slightly between Xray Cloud, Xray Data Center and different versions, so follow the labels on your screen.

5.1 Creating a Test issue

1. In Jira, click **Create**.
2. Select the **Project** and set **Issue Type** to **Test**.
3. Enter a **Summary** that states the behaviour being verified, for example "Login with valid credentials".
4. Select the **Test Type**: Manual, Cucumber or Generic. Depending on the version and screen configuration, this is chosen on the create screen or on the issue afterwards.
5. Complete the other fields your team uses, such as Priority, Components, Labels and Description.
6. Click **Create**, then open the new Test to add its steps.

5.2 Writing a proper test case

A good test case is understandable by someone who did not write it. Use these rules:

- Give the Test one clear objective. A Test that verifies login, search and checkout together is hard to diagnose when it fails.
- Start the summary with the behaviour or outcome, not with "Test".
- State the preconditions separately from the steps.
- Write each step as one action by the tester, with the data to use.
- Give each step an expected result that can be observed and judged as pass or fail.
- Avoid vague wording such as "works correctly" or "as expected".

5.3 Adding test steps

Manual Tests are built from steps. Each step has three fields: **Action** (what the tester does), **Data** (the input or values to use) and **Expected Result** (what should happen). [12]

1. Open the Test issue.
2. In the test steps section, choose the option to add a step.
3. Enter the Action, Data and Expected Result for the step.
4. Repeat for each step. Steps can be reordered or edited later.

5.4 Defining expected results

The expected result is what makes a Test verifiable, because the result of a step is decided by comparing the actual result with it. A good expected result names the visible outcome, for example "The dashboard page is displayed and the header shows the customer name". A weak one is "Login works". For the Test as a whole, the final step should check the main outcome of the Test.

5.5 Linking Tests to requirements or user stories

Linking is what creates requirement coverage. First, an administrator must configure the project so that the chosen issue types (for example Story) are treated as requirements for coverage. [6] Then:

1. Open the Test and use **Link issue** (or the equivalent link control).
2. Choose the link type used for coverage. By default this is the Xray "Test" link: the Test **tests** the story.
3. Select the story or requirement, for example SHOP-12, and save.
4. Open the story and check that the Test now appears in its test coverage information.

The link can also be created from the story side ("is tested by"). The result is the same.

5.6 Creating a Test Set

1. Click **Create**, select the project and set **Issue Type** to **Test Set**.
2. Enter a Summary that describes the purpose, for example "Smoke - Core Purchase Flow".
3. Add a Description if useful, then click **Create**.

5.7 Adding Tests to a Test Set

1. Open the Test Set.
2. In the Tests section, choose the option to add Tests (for example **Add Test**). [12]
3. Select existing Tests, or search for them, and confirm.
4. Check that the Tests are listed. A Test can be added to more than one Test Set.

5.8 Creating a Test Plan

1. Click **Create**, select the project and set **Issue Type** to **Test Plan**.
2. Enter a Summary, for example "ShopEasy Release 1.0 Test Plan".
3. Set the **Fix Version** if your team scopes plans by release.
4. Click **Create**.

The Test Plan issue type must be available in the project's issue type scheme, otherwise it will not appear in the list. [4]

5.9 Adding Tests to a Test Plan

1. Open the Test Plan.
2. In the Tests section, choose the option to add Tests, or to add a Test Set, so that all Tests from the set are added.
3. Select the Tests or the Test Set and confirm.
4. Review the list. The Test Plan now shows the latest result of each Test once executions are linked to it.

5.10 Sample test case formats

Manual test case. Fields are shown as they would be filled in on a Test issue.

Issue type	Test
Summary	Login with valid credentials
Test Type	Manual
Priority	High
Linked requirement	SHOP-12 User Login (the Test "tests" the story)
Precondition	SHOP-41 A registered, active customer account exists

Step	Action	Data	Expected Result
1	Open the login page	URL of the staging site	The login form with email and password fields is displayed
2	Enter the registered email address	customer01@example.com	The email appears in the field
3	Enter the password	The password of customer01	The password is masked
4	Click Login	-	The customer dashboard opens and the header shows the customer name

Cucumber test case. A Cucumber Test holds a Gherkin scenario instead of steps:

```
Feature: Product search
  Scenario: Search by exact product name
    Given the product catalogue contains the product "Wireless Mouse"
    When the customer searches for "Wireless Mouse"
    Then the search results list the product "Wireless Mouse"
```

6. Test Execution in Xray

6.1 What is Test Execution?

Test execution is the act of running Tests and recording the results. In Xray it happens inside a **Test Execution** issue. Each Test added to the issue receives a **Test Run**, and the tester works through the Test Runs one by one. Tests are always executed in the context of a Test Execution, even for a quick, one-off run. [2]

Three levels of status are involved. A **step status** is the result of one step. A **Test Run status** is the result of one Test in one execution and is calculated from its step statuses. The **overall status of the Test Execution** is updated as each Test Run changes, so it shows how many Tests passed, failed, are running or are still waiting. [12]

6.2 Creating a Test Execution

1. Click **Create**, select the project and set **Issue Type** to **Test Execution**.
2. Enter a Summary that identifies the round, for example "Release 1.0 Full Regression (Chrome, Staging)".
3. Complete the Xray fields your team uses, such as Fix Version, Revision (the build or commit being tested), Begin and End date, and Test Environments. [12]
4. Optionally associate a Test Plan, so the results count towards that plan.
5. Click **Create**.

6.3 Adding Tests to an execution

1. Open the Test Execution.
2. In the Tests section, choose the option to add Tests. Tests can be selected individually, from a Test Set, or from a Test Plan.
3. Confirm. Xray creates a Test Run in status TODO for each Test.

If Tests are later added to a Test Set, they can be added to an execution that was created from it by importing the set again; only the new Tests are added.

6.4 Executing manual tests

1. Open the Test Execution and click the execute control next to a Test to open its Test Run screen.
2. Start the Test Run. Its status changes to EXECUTING, and the timer in the execution screen starts; the timer pauses when the run reaches a final status. [2]
3. Review the Preconditions and make sure they are satisfied.
4. Perform each step and compare what happened with the Expected Result.
5. Set the status of each step, and then of the Test Run, as described in the next sections.

6.5 Recording actual results

For each step, record what actually happened in the actual result or comment field, especially when it differs from the expected result. Attach evidence such as screenshots, logs or screen recordings to the step or to the Test Run. Good actual results make a defect reproducible without asking the tester to repeat the run.

6.6 Updating test status

Set each step to PASS or FAIL as you go. With the built-in statuses, Xray calculates the status of the Test Run from its steps: if any step fails the Test Run is FAIL, and it is PASS only when all steps pass. You can also set the Test Run status directly, for example to ABORTED if the run could not be completed. [3]

6.7 Handling failed test cases

1. Record the actual result and attach evidence for the failed step.
2. Confirm the failure is reproducible and not caused by the environment or test data. If it is an environment problem, correct it and re-run instead of raising a defect.
3. Create or link a defect as described in 6.8.
4. Leave the Test Run as FAIL. Do not change it to PASS until the fix has been verified.
5. After the fix is delivered, run the Test again, in the same execution if your team's process allows it or in a new one. The latest run decides the Test's current status. [3]

6.8 Associating bugs with failed tests

From the Test Run screen, use the defect option to create a new Bug, or to link an existing one. When the Bug is created from the Test Run, Xray keeps it linked to that run, which makes it visible in the Test Execution and in the traceability and plan reports as a linked defect. [4][6] Check that the Bug contains the steps, data, environment and evidence from the run. Also link the Bug to the affected requirement if your team wants it visible there.

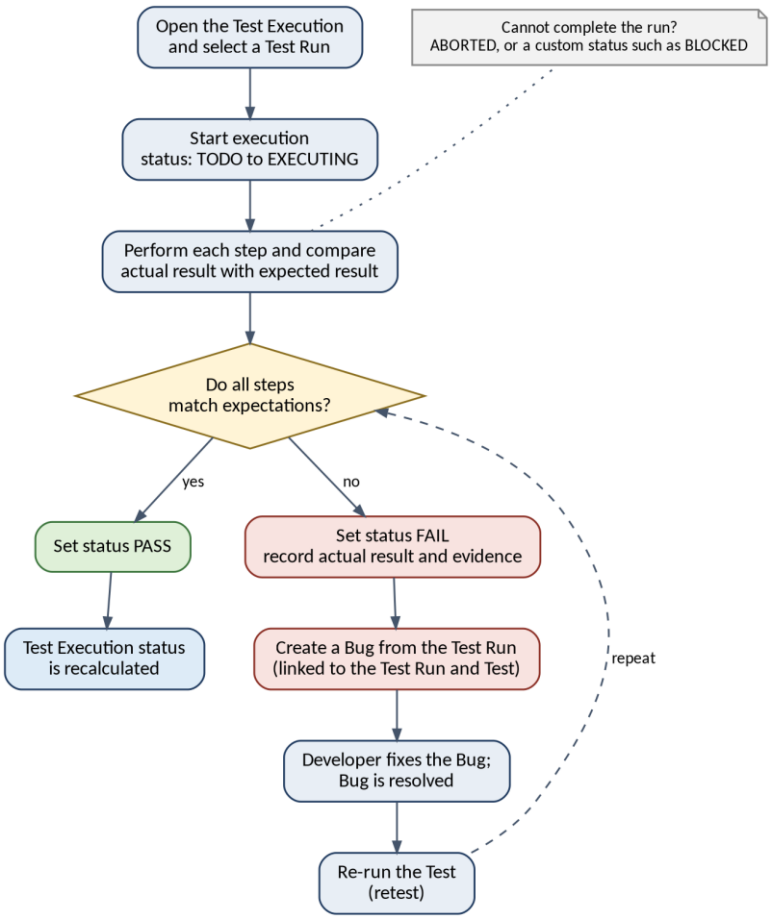


Figure 3: Test execution and defect reporting workflow

6.9 Execution statuses

Xray ships with built-in statuses and lets administrators add custom ones. The table shows the built-in statuses, whether each is final, and the requirement coverage status it maps to by default. [3]

Status	Meaning	Final?	Coverage status	Built in?
TODO	The Test has not been run yet	No	NOTRUN	Yes
EXECUTING	The Test is being run now	No	NOTRUN	Yes
PASS	The Test passed (some screens show PASSED)	Yes	OK	Yes
FAIL	The Test failed (some screens show FAILED)	Yes	NOK	Yes
ABORTED	The run was stopped before it could finish	Yes	NOTRUN	Yes
BLOCKED	The run cannot continue because of an outside problem	Set when created	Set when created	No, a custom status

Version and configuration note. BLOCKED is a very common status in test management, but it is not one of Xray's native statuses. An administrator can create it, or any other custom status, and must choose which coverage status it maps to. In Xray Cloud this is done in the Jira Global Settings (Apps section, Xray Test Statuses) [8]; in Data Center it is under Manage Test Statuses in the Xray administration [9]. The statuses available to testers therefore depend on the Xray version and on how your administrator has configured it. Native statuses cannot be changed or deleted.

7. Requirement Traceability and Test Coverage

7.1 What is requirement traceability?

Requirement traceability is a record of which tests verify which requirements, and what the results are. With it, the team can answer two questions: "which tests prove that this requirement works?" and "which requirement is affected by this failing test?". It also supports audits in regulated industries, where evidence that each requirement was verified is mandatory.

7.2 How Xray links requirements with Tests

Xray uses an issue link between a Test and the requirement. By default the link is named "Test": the Test **tests** the requirement, and the requirement **is tested by** the Test. The project must be configured so that the requirement issue types (such as Story or Epic) are included in requirement coverage. [6] Once linked, the requirement carries a coverage status calculated from the latest result of each linked Test. The default status is evaluated in a context, which can be a version, a Test Plan or a Test Environment. [3]

Coverage status	Meaning	Typical action
OK	All Tests linked to the requirement have passed	None
NOK	At least one linked Test has failed	Investigate and fix the defect
NOTRUN	No linked Test has failed, but at least one is TODO or ABORTED	Complete the pending runs
UNKNOWN	At least one linked Test is in an unknown status and none has failed	Check the custom status mapping
UNCOVERED	The requirement has no Tests linked to it	Design Tests for it

7.3 Identifying requirements without test coverage

1. Open the **Test Coverage** report (in the Xray reports for the project, or under the Jira Reports menu) and select the project, version or Test Plan.
2. Look for requirements with status UNCOVERED. They have no linked Tests.
3. Open the **Traceability** report to see each requirement with its Tests, Test Runs and defects, which also shows requirements that have Tests but no results yet. [6]
4. Open an individual story and check its test coverage information.
5. Create and link Tests for each uncovered requirement, then review the report again.

Xray also provides JQL functions for searching Tests by their relationship to other Xray issues. Which functions are available depends on the edition and version, so check the JQL documentation for your instance.

7.4 Analysing test coverage

Coverage analysis looks at two things. **Breadth** is the share of requirements that have at least one Test. **Depth** is the share of requirements whose Tests passed. A project can have 100% breadth and still have poor depth if many Tests fail. Coverage should also be read in context: a requirement may be OK for one Test Plan or environment and NOK for another. [3]

Coverage status shows that Tests exist and have passed, not that the Tests are good. A requirement covered by one superficial Test will still show OK. Review the quality of the Tests as well as the status.

7.5 Why traceability is important

- It shows gaps early, before a requirement ships untested.
- It lets a failing Test be tied back to the affected requirement, so product owners can judge the impact.
- It limits regression testing after a change: when a requirement changes, the linked Tests are the ones to review.
- It provides evidence for audits and customer sign-off.

7.6 Practical example: login module

The login module of the shop has two stories. SHOP-12 "User Login" has three Tests. SHOP-18 "Reset forgotten password" has none.

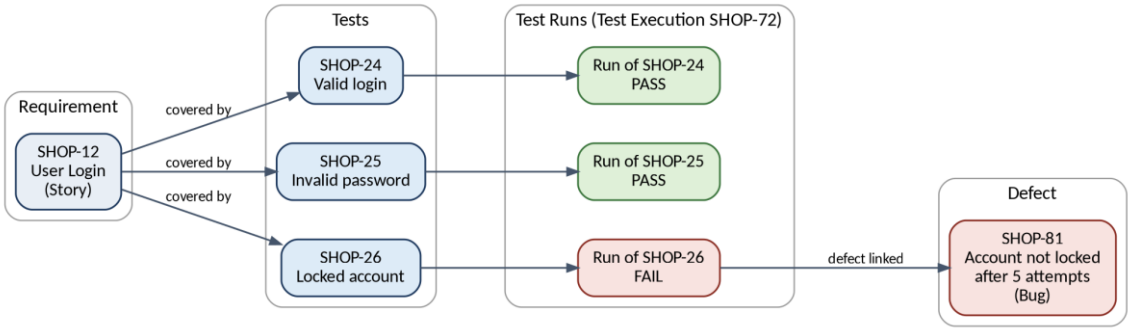


Figure 4: Requirement-to-test traceability for the login story

Requirement	Linked Tests	Latest results	Coverage status
SHOP-12 User Login	SHOP-24, SHOP-25, SHOP-26	PASS, PASS, FAIL (SHOP-81 raised)	NOK
SHOP-18 Reset forgotten password	None	-	UNCOVERED

SHOP-12 is NOK because one linked Test has failed. The team fixes SHOP-81, re-runs SHOP-26 and, when it passes, SHOP-12 becomes OK. SHOP-18 is UNCOVERED, so the QA engineer creates Tests for the reset-password flow and links them to the story.

8. Xray Reports and Dashboards

Xray reports can be opened from the Xray reports icon in the project sidebar or from the Jira Reports menu. Report names, filters and some options differ between Xray Cloud and Data Center and between versions. The table lists the important reports and notes where availability differs. Do not assume that every report is available in every edition; open the Reports area of your instance to see what is installed. [4][5]

Report	Purpose	Typical question answered	Availability
Test Coverage Report	Shows the coverage status of requirements, for example as a bar chart, filtered by version, Test Plan or environment	Which requirements are OK, NOK, NOTRUN or UNCOVERED?	Cloud and Data Center
Requirement Traceability Report	Follows requirements forwards and backwards through Tests, Test Runs and defects. Requires requirement coverage to be enabled for the project. [6]	Which tests and defects affect this requirement?	Cloud and Data Center
Test Execution reporting	Data Center: Test Executions Report, listing executions with counts per run status and test type, progress and linked defects. [5] Cloud: Test Executions List and Test Runs List reports [10], and a Test Execution List gadget for dashboards [11]	How did each execution go, and how do cycles compare?	Differs; the Executions Report is documented for Data Center
Test Plan Report	Data Center: Test Plans Report, showing number of executions and runs, Tests by status, progress and linked defects. It needs the Test Plan issue type in the project. Selecting several projects needs Xray Enterprise. [4] Cloud: Test Plans List and Test Plans Metrics reports	How far has this plan progressed?	Differs by edition
Overall testing progress	Progress of a Test Plan is the percentage of its Tests that have a final status. [4] Dashboards and gadgets built on JQL filters give a project-wide view	Are we on track for the release date?	Cloud and Data Center; gadgets vary

8.1 How QA teams use the reports

- **Daily monitoring:** the test lead checks Test Plan progress and the list of failed Tests to decide what needs attention that day.
- **Release decisions:** before a release, the team checks that the Test Coverage report shows no UNCOVERED requirements and that open defects from failed runs are understood.
- **Defect focus:** failed runs with their linked defects show where developers should concentrate.
- **Process improvement:** comparing executions across sprints shows whether pass rates are improving and which areas keep failing.
- **Stakeholder updates:** a dashboard with a Test Execution list gadget gives managers a view without opening each issue. [11]

Some Xray reports do not offer an export option, and CI-heavy projects can generate thousands of Test Runs, which can make detailed reports slow. In those cases use filters and summary views. [6]

9. Real-Time Practical Case Study: E-Commerce Application

This case study follows a QA engineer testing release 1.0 of "ShopEasy", an online shop, with Xray. The Jira project key is SHOP. Issue keys are illustrative. The six modules are User Registration, User Login, Product Search, Add to Cart, Checkout and Payment.

9.1 Project setup

The Jira administrator enables the Xray issue types for the SHOP project and configures Story as a requirement issue type for coverage. The QA engineer then works through the workflow of Section 4.

9.2 Sample Jira user stories

Key	Story	Acceptance criteria (summary)	Module
SHOP-11	As a visitor, I want to register an account so that I can place orders	Valid details create an account; invalid or duplicate email is rejected	User Registration
SHOP-12	As a customer, I want to log in so that I can see my orders	Valid credentials open the dashboard; wrong password is rejected; account locks after five failed attempts	User Login
SHOP-13	As a customer, I want to search for products so that I can find what I need	Matching products are listed; a clear message appears when none match	Product Search
SHOP-14	As a customer, I want to add products to my cart so that I can buy several items	Products can be added; quantities can be changed; totals update	Add to Cart
SHOP-15	As a customer, I want to check out so that I can confirm my order	Saved address can be used; mandatory address fields are validated	Checkout
SHOP-16	As a customer, I want to pay securely so that my order is confirmed	Valid payment confirms the order; declined payment gives a clear message and allows retry	Payment

9.3 Preconditions

Key	Precondition	Used by
SHOP-41	A registered, active customer account exists (customer01@example.com)	SHOP-23 to SHOP-26
SHOP-42	The customer is logged in and the cart contains one in-stock product	SHOP-31 to SHOP-35
SHOP-43	The catalogue contains at least 20 active products	SHOP-27 to SHOP-30

9.4 Sample test cases

The QA engineer creates fifteen Tests and links each to its story. SHOP-27 is a Cucumber Test; the others are Manual.

Key	Summary	Story	Type	Priority	Precondition
SHOP-21	Register with valid details	SHOP-11	Manual	High	None
SHOP-22	Registration rejects an invalid email format	SHOP-11	Manual	Medium	None
SHOP-23	Registration rejects an already registered email	SHOP-11	Manual	Medium	SHOP-41
SHOP-24	Login with valid credentials	SHOP-12	Manual	High	SHOP-41
SHOP-25	Login fails with a wrong password	SHOP-12	Manual	High	SHOP-41
SHOP-26	Account locks after five failed attempts	SHOP-12	Manual	Medium	SHOP-41
SHOP-27	Search by exact product name	SHOP-13	Cucumber	High	SHOP-43
SHOP-28	Search with no matching results	SHOP-13	Manual	Medium	SHOP-43
SHOP-29	Add one product to the cart	SHOP-14	Manual	High	SHOP-43
SHOP-30	Change the quantity in the cart	SHOP-14	Manual	Medium	SHOP-43
SHOP-31	Checkout with a saved address	SHOP-15	Manual	High	SHOP-42
SHOP-32	Checkout rejects a missing postcode	SHOP-15	Manual	Medium	SHOP-42
SHOP-33	Pay with a valid credit card	SHOP-16	Manual	High	SHOP-42
SHOP-34	Pay with a declined card	SHOP-16	Manual	High	SHOP-42
SHOP-35	Pay using UPI in the sandbox gateway	SHOP-16	Manual	Medium	SHOP-42

Steps of Test SHOP-34 "Pay with a declined card":

Step	Action	Data	Expected Result
1	Open the payment page from the checkout	Cart with one product	The payment form is displayed with the order total
2	Enter the card details of the test card that the gateway declines	Declined test card number from the gateway documentation	The card number is accepted by the form
3	Click Pay	-	A message states that the card was declined and no order is created
4	Check the cart	-	The cart still contains the product and the customer can try another payment method

9.5 Test Sets

Key	Test Set	Tests
SHOP-51	Smoke - Core Purchase Flow	SHOP-24, SHOP-27, SHOP-29, SHOP-31, SHOP-33
SHOP-52	Regression - Account Management	SHOP-21 to SHOP-26
SHOP-53	Regression - Purchase Flow	SHOP-27 to SHOP-35

9.6 Test Plan

Test Plan **SHOP-61 "ShopEasy Release 1.0 Test Plan"** (fix version 1.0) is created, and the Tests of Test Sets SHOP-52 and SHOP-53 are added, so the plan tracks all fifteen Tests.

9.7 Test Executions

Key	Test Execution	Tests taken from	Environment and revision
SHOP-71	Release 1.0 Smoke (Chrome, Staging)	Test Set SHOP-51 (5 Tests)	Staging, Chrome, build 1.0.3
SHOP-72	Release 1.0 Full Regression (Chrome, Staging)	Test Plan SHOP-61 (15 Tests)	Staging, Chrome, build 1.0.4

Both executions are associated with Test Plan SHOP-61. The smoke run on build 1.0.3 passes all five Tests. The full regression on build 1.0.4 gives the results below.

9.8 Sample execution results (SHOP-72)

Test	Summary	Status	Actual result or comment	Defect
SHOP-21	Register with valid details	PASS	Account created, confirmation shown	
SHOP-22	Invalid email format rejected	PASS	Inline error shown	
SHOP-23	Duplicate email rejected	PASS	Message shown, no second account	
SHOP-24	Login with valid credentials	PASS	Dashboard opened	
SHOP-25	Wrong password rejected	PASS	Error shown, user stays on login page	
SHOP-26	Account locks after five failures	FAIL	Sixth attempt with the correct password still logged in; no lock message	SHOP-81
SHOP-27	Search by exact name	PASS	Product listed first	
SHOP-28	Search with no results	PASS	No-results message shown	
SHOP-29	Add one product	PASS	Cart count shows 1	
SHOP-30	Change quantity	PASS	Total recalculated	

Test	Summary	Status	Actual result or comment	Defect
SHOP-31	Checkout with saved address	PASS	Order summary shown	
SHOP-32	Missing postcode rejected	TODO	Not yet run	
SHOP-33	Pay with valid card	PASS	Order confirmed	
SHOP-34	Pay with declined card	FAIL	Generic error shown and the cart was emptied	SHOP-82
SHOP-35	Pay using UPI (sandbox)	ABORTED	Sandbox gateway timed out; run stopped	

Totals for SHOP-72: 11 PASS, 2 FAIL, 1 ABORTED, 1 TODO. If the team has created a custom BLOCKED status (see 6.9), SHOP-35 could be marked BLOCKED instead of ABORTED.

9.9 Defect creation and linking

When SHOP-26 failed, the tester recorded the actual result and attached a screenshot. From the Test Run, the tester created Bug **SHOP-81**, which stays linked to the run of SHOP-26 in SHOP-72. The same was done for SHOP-34, creating Bug **SHOP-82**. The tester also linked each Bug to its story with a normal Jira link.

Key	Summary	Priority	Raised from	Affected story
SHOP-81	Account is not locked after five failed login attempts	High	Test Run SHOP-26 in SHOP-72	SHOP-12
SHOP-82	Declined card shows a generic error and empties the cart	High	Test Run SHOP-34 in SHOP-72	SHOP-16

9.10 Requirement traceability result

Story	Linked Tests	Latest results	Coverage status
SHOP-11 Registration	3	3 PASS	OK
SHOP-12 Login	3	2 PASS, 1 FAIL (SHOP-81)	NOK
SHOP-13 Product Search	2	2 PASS	OK
SHOP-14 Add to Cart	2	2 PASS	OK
SHOP-15 Checkout	2	1 PASS, 1 TODO	NOTRUN
SHOP-16 Payment	3	1 PASS, 1 FAIL (SHOP-82), 1 ABORTED	NOK

9.11 Complete workflow summary

1. The product owner writes stories SHOP-11 to SHOP-16.
2. The QA engineer creates Preconditions SHOP-41 to SHOP-43.
3. The QA engineer writes Tests SHOP-21 to SHOP-35, links them to the stories and to the Preconditions.
4. Tests are grouped into Test Sets SHOP-51, SHOP-52 and SHOP-53.
5. Test Plan SHOP-61 defines the scope of release 1.0.
6. Test Executions SHOP-71 (smoke) and SHOP-72 (regression) are created and associated with the plan.
7. Testers run the Tests, record statuses and actual results, and attach evidence.
8. Bugs SHOP-81 and SHOP-82 are raised from the failed runs and assigned to developers.
9. The test lead reviews the Test Plan, the coverage status of each story and the traceability result.
10. Decision: release 1.0 is not ready. Login and Payment are NOK, Checkout is NOTRUN and one run was aborted. After the bugs are fixed, SHOP-26 and SHOP-34 are re-run, SHOP-32 and SHOP-35 are executed, and the coverage is reviewed again.

10. Xray vs Traditional Jira Testing

A team without a test management app can still track tests in Jira by creating ordinary issues (for example a "Task" or a custom "Test Case" type) and using fields, links and comments. This works for very small projects but lacks the structure that testing needs.

Capability	Ordinary Jira issues	Xray
Test case structure	Free-text description; steps are written by hand	Dedicated Test issue with Test Types and structured steps (action, data, expected result)
Reuse across releases	Copy the issue or reopen it, which loses history	One Test is reused in many executions; each run keeps its own result
Execution results	One Jira status for the whole issue	A Test Run per Test per execution, with step results, actual results and evidence
Execution history	Overwritten by workflow transitions or kept in comments	Every Test Run is kept, so the history of a Test can be reviewed
Grouping and planning	Labels, epics or filters	Test Sets, Test Repository, Test Plans and Test Executions
Requirement coverage	Manual links; no calculation	Test link plus calculated coverage status (OK, NOK, NOTRUN, UNKNOWN, UNCOVERED)
Traceability	Manual and hard to maintain	Traceability report from requirement to Tests, runs and defects

Capability	Ordinary Jira issues	Xray
Defects	Linked manually; link to the failing step is lost	Bug created from a failed Test Run and kept linked to it
Reports	Generic Jira reports and filters	Test-specific reports and gadgets
Automation	No standard way to bring results in	Import of results from CI tools such as JUnit, TestNG and Cucumber

10.1 Advantages of Xray for professional QA teams

- Testing information is structured, reusable and searchable.
- Requirement coverage and test progress are measured instead of estimated.
- Audit evidence exists for every run: who ran it, when, on which build and with what result.
- Manual and automated results are visible in the same reports.
- Existing Jira skills, workflows and permissions can be reused, so the team works in one tool.

Xray is a licensed app and needs an administrator to configure it. Small teams with few test cases may decide that this overhead is not justified; for professional QA teams the benefits above normally outweigh it.

11. Best Practices for Using Xray

Practice	What to do
Writing effective test cases	One objective per Test; clear summary; one action per step; observable expected results; test data stated in the Data field; avoid vague wording.
Organising Tests into Test Sets	Create sets for lasting purposes such as smoke, regression and module. Use clear naming (for example "Smoke - Core Purchase Flow"). Do not create a set for every sprint; use Test Plans and executions for that. Choose either Test Sets or Test Repository folders as the main structure, and keep it consistent.
Maintaining Test Plans	One plan per release or cycle, with a fix version. Add Tests when scope changes, remove Tests that are out of scope, and close the plan when the release is done.
Recording execution results accurately	Update the status as the step is done, not at the end of the day. Always write an actual result for failures and attach evidence. Record the build or revision and the environment on the execution.
Maintaining requirement traceability	Link every Test to at least one requirement when it is created. Review the Test Coverage report each sprint and act on UNCOVERED requirements. Update links when requirements change.
Avoiding duplicate test cases	Search by keyword or JQL before creating a Test. Agree naming conventions. Review the test library regularly and merge or retire overlapping Tests.
Linking defects correctly	Create defects from the failed Test Run so the link is automatic. Include steps, data, environment and evidence. Link the defect to the affected story. Re-run the Test to verify the fix.
Maintaining reusable test cases	Write Tests that do not depend on a release number. Put shared setup in Preconditions. Version-control behaviour changes by updating the Test and noting the change. Use a workflow to review Tests before they are approved. Retire obsolete Tests instead of deleting them, so history is kept.

12. Conclusion

Xray turns Jira into a test management tool. It adds Test, Test Set, Test Plan, Test Execution and Precondition issue types, a Test Run for each execution of a Test, calculated coverage statuses and test-specific reports, while the requirements and defects remain ordinary Jira issues. The relationships between these items are associations, mostly many-to-many, and not a strict hierarchy.

Used well, Xray improves a QA workflow in four ways. Test cases are written once and reused. Results are recorded per step and per run with history. Requirements are traced to Tests, results and defects. Progress and coverage are visible to the whole team through reports and dashboards.

For an SDET or QA automation engineer, knowledge of Xray is valuable for three reasons. Many organisations that use Jira use Xray for test management. Automated test results from CI pipelines are imported into Xray and appear next to manual results, so an SDET is often responsible for that integration. And understanding traceability and coverage helps an engineer design tests that answer real questions about release readiness, rather than only increasing the number of tests.

Exact menu names, report names and available statuses depend on the Xray edition, version and administrator configuration. Always confirm details in the documentation for the version in use.

References

[1] Xray documentation, home page. https://docs.getxray.app/

[2] Xray Data Center documentation, Execute Tests. https://confluence.getxray.app/display/XRAY/Execute+Tests

[3] Xray Cloud documentation, Understanding the calculation of coverage status and the status of Tests. https://confluence.getxray.app/display/XRAYCLOUD/Understanding+the+calculation+of+coverage+status+and+the+status+of+Tests

[4] Xray Data Center documentation, Test Plans Report. https://confluence.getxray.app/display/XRAY/Test+Plans+Report

[5] Xray Data Center documentation, Test Executions Report. https://confluence.getxray.app/display/XRAY/Test+Executions+Report

[6] Xray documentation (version 3.1), Traceability Report. https://confluence.getxray.app/display/XRAY31/Traceability+Report

[7] Xray documentation (version 3.6.0), Test Process. https://confluence.getxray.app/display/XRAY360/Test+Process

[8] Xray Cloud documentation, Global Settings: Test Statuses. https://docs.getxray.app/display/XRAYCLOUD/Global+Settings%3A+Test+Statuses

[9] Xray Data Center documentation, Manage Test Statuses. https://docs.getxray.app/display/XRAY/Manage+Test+Statuses

[10] Xray Cloud documentation, Test Runs List Report. https://docs.getxray.app/display/XRAYCLOUD/Test+Runs+List+Report

[11] Xray Cloud documentation, Test Execution List Gadget. https://confluence.getxray.app/display/XRAYCLOUD/Test+Execution+List+Gadget

[12] Atlassian, Jira Xray integration: manage test cases (tutorial). https://www.atlassian.com/devops/testing-tutorials/jira-xray-integration-manage-test-cases